

Manual do Usuário - AGRUPA

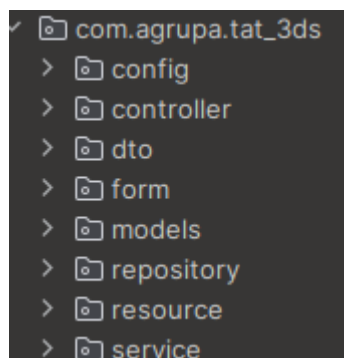
Back-end

1. Visão Geral

Este manual tem como objetivo orientar a execução do Back-end e Banco de Dados do sistema de Gerenciamento de Grupos (AGRUPA) e contribuir com informações sobre o projeto, o mesmo foi desenvolvido em Java + Springboot, focando na gestão de grupos e organização na sala de aula.

2. Estrutura do Projeto

O projeto está estruturado de maneira a unificar as classes baseando-se em suas funcionalidades, como controller, models, repositórios, services e DTOs entre outros:



- **config:** Contém as classes de configuração do sistema. É onde se define como a aplicação interage com o ambiente externo e infraestrutura.
 - *Exemplo:* SecurityConfig (regras de autenticação), SwaggerConfig (documentação da API), CorsConfig.
- **controller:** Ponto de entrada das requisições HTTP. Os controllers validam parâmetros básicos e delegam a execução para a camada de serviço mas não devem possuir as regras de negócio.
 - *Exemplo:* GrupoUsuarioController mapeando GETs, POSTs e outros.
- **dto (Data Transfer Object):** Classes usadas exclusivamente para trafegar dados entre a API (Controllers) e os models. Servem para ocultar dados sensíveis dos models e permitir a manipulação dos dados de forma mais eficaz.
 - *Exemplo:* GrupoInformacoesDTO (retornando apenas as informações necessárias).

- **form**: Uma variação de DTOs, geralmente focada na **entrada** de dados. Classes que representam o corpo de uma requisição que altera o estado do sistema e contêm anotações de validação (ex: @NotNull, @Email).
 - *Exemplo*: UsuarioCreateForm (validando se a senha informada atende aos requisitos).
- **models**: Onde residem as entidades de domínio, geralmente mapeadas diretamente para as tabelas do banco de dados (usando JPA/Hibernate, por exemplo).
 - *Exemplo*: Usuario (com atributos como id, nome, email, senha e mapeamentos como @Entity).
- **repository**: Interfaces ou classes responsáveis pela comunicação com o banco de dados. Isolam os detalhes da persistência do resto da aplicação.
 - *Exemplo*: UsuarioRepository (herdando de JpaRepository para operações de salvar, buscar, deletar).
- **resource**: Este pacote pode ser dúbio dependendo do padrão da equipe. Em alguns projetos, representa recursos estáticos (arquivos de propriedades, templates HTML). Em outros (como no padrão JAX-RS), atua como sinônimo de controller para APIs REST. **É crucial clarificar no seu guia a função exata deste pacote no contexto do seu projeto.**
- **service**: O coração da aplicação. Onde as regras de negócio reais vivem. Os serviços orquestram chamadas aos repositórios e outras integrações.
 - *Exemplo*: UsuarioService contendo um método registrarNovoUsuario() que verifica se o e-mail já existe antes de salvar no repositório.

3. Conceitos e ferramentas

3.1 Requisições HTTP

Uma requisição HTTP é uma mensagem enviada para um servidor de um determinado cliente, nela contém as informações necessárias para ser realizada a operação solicitada (body ou corpo), assim obtemos a definição do tipo de requisição, o endereço do servidor. Os métodos principais relacionados a requisições HTTP são:

- GET: Requisição busca de dados sem alteração do estado.
- POST: Envia dados ao servidor para criar um novo recurso.
- PUT: Atualiza dados que já existem dentro do servidor.
- DELETE: Remove dados.

Nosso projeto web é totalmente estruturado baseando-se em protocolos HTTP, funcionando de maneira harmoniza.

3.2 Banco de dados em memória

Se tratando de uma aplicação, que não necessita de armazenamento de dados e de informação a longo prazo, foi utilizado banco em memória, esta escolha não está relacionada somente a isso mas também unificando a praticidade ao rodarmos a aplicação, utilizando o banco HSQLDB ou também conhecido como HyperSQL DataBase podemos obter o melhor dos mundos, armazenando a informação dos usuários e dos alunos que estão na sala e ao mesmo tempo possibilitando o registro de dados em tempo real, contribuindo para a geração de relatórios e a criação dos grupos.

3.3 Data Transfer Object

Conhecidos como DTOs ou Data Transfer Object, o DTO é uma boa prática utilizada no desenvolvimento para reduzir a exposição de dados e facilitar a manipulação de informações entre o back-end e o front-end, ao utilizando o DTO diminuimos a exposição de camadas mais internas ficarem expostas instanciando uma classe com os atributos necessários e a utilizando para envio de informações ao front-end.

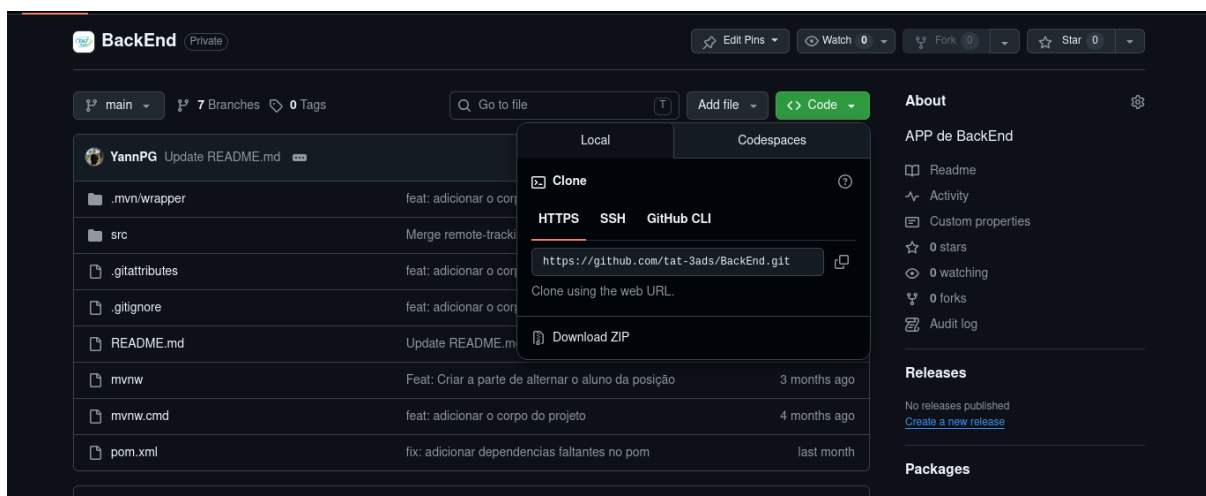
4. Pré-requisitos

4.1 Clonar o projeto

Para conseguirmos rodar o projeto primeiro precisaremos clonar o projeto, para isso precisamos do git instalado em nossa máquinas.

- Link para download do git: <https://git-scm.com/install/>

Com o git instalado poderemos dar continuidade a próxima etapa, clonar o projeto, para isso precisaremos acessar o projeto dentro do github e copiar a url.



Agora em nosso terminal, já na pasta na qual iremos clonar nosso projeto iremos clonar.

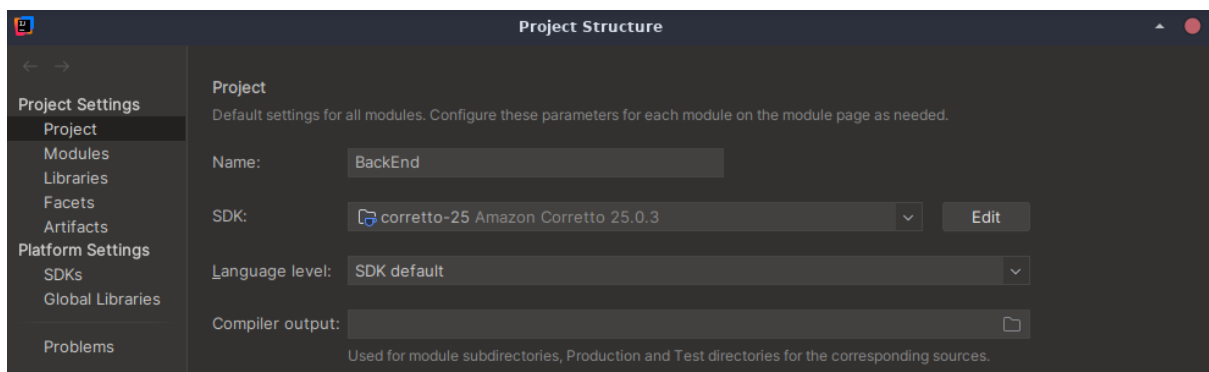
- `git clone https://github.com/tat-3ads/BackEnd.git`

caso seja necessário faça as configurações necessárias para clonar o projeto do git.

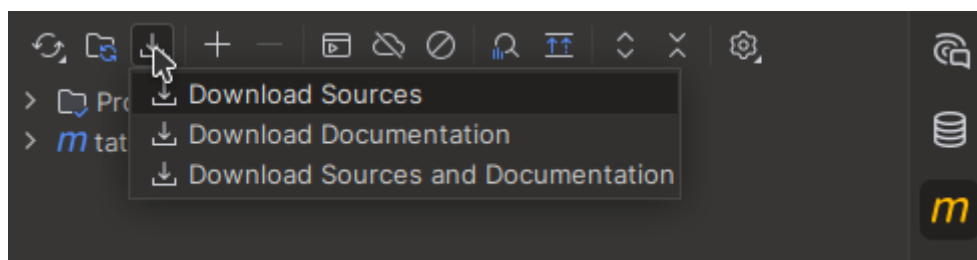
4.2 Configurar o ambiente

Para rodarmos o back-end, é necessário garantir que o ambiente Java esteja configurado corretamente

- Ter o Java instalado.
- Java (versão 25).
 - o Verifique a versão do Java evitar erros de versão, no exemplo abaixo estamos utilizando intellij mas podem ser usadas outras ferramentas (em Project Structure).



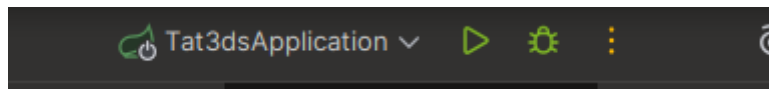
- Atualização e Baixar as dependências. (sempre que algo for adicionado ou adicionado ao projeto é de bom tom atualizarmos e baixarmos novamente as dependências, garantindo que o projeto rode corretamente).



5. Como rodar o projeto

Para rodar o projeto primeiramente siga as recomendações do passo 4, após isso verifique.

- As portas estão liberadas para rodar o projetos, libere as portas padrões para conseguirmos prosseguir.
- Para conseguirmos rodar o projeto, primeiramente é necessário selecionar o arquivo Tat3dsApplication para rodar o projeto.



Caso selecionado selecione o arquivo, como no exemplo abaixo

